

Práctico

Modelos Dinámicos – Ingeniería Inversa

Diagramas de Interacción en UML: Secuencia y Comunicación

1- Dado el siguiente código Java para un applet:

```
/* @author Scott Violet */

public class SampleTreeModel extends DefaultTreeModel {

    public SampleTreeModel(TreeNode newRoot) { super(newRoot); }

    public void valueForPathChanged(TreePath path, Object newValue) {
        DefaultMutableTreeNode aNode =
            (DefaultMutableTreeNode)path.getLastPathComponent();
        SampleData sampleData = (SampleData)aNode.getUserObject();

        sampleData.setString((String)newValue);
        if (this.isColorGreen) sampleData.setColor(Color.green);
        else sampleData.setColor(Color.black);

        nodeChanged(aNode);
    }
}
```

Construir un modelo de diseño dinámico con un Diagrama de Comunicación para la invocación del mensaje “valueForPathChanged”.

Considerar en el modelo los siguientes elementos según corresponda:

- a. considere los retornos y los valores de retornos de los mensajes.
- b. Condiciones e iteraciones.

2- Dado el siguiente código Java para un applet que responde a eventos de checkboxes, construya el modelo de diseño dinámico a través de un Diagrama de Comunicación correspondiente. Tenga en cuenta que por el método init se iniciará la ejecución.

```
import java.awt.*;
import java.applet.*;

public class CheckboxApplet2 extends Applet
{
    Checkbox checkbox1; Checkbox checkbox2; Checkbox checkbox3;

    public void init() //se redefine el método init
    {
        checkbox1 = new Checkbox("Option 1", null, true);
        checkbox2 = new Checkbox("Option 2", null, false);
        checkbox3 = new Checkbox("Option 3", null, false);
        add(checkbox1);
        add(checkbox2);
        add(checkbox3);
    }
}
```

```

    }

    public boolean action(Event evt, Object arg) //se redefine el método action {
        if (evt.target instanceof Checkbox) // si Checkbox causó el evento
            ChangeLabel(evt);
        repaint(); // Fuerza a Java a repintar el display del applet return true;
    }

    protected void ChangeLabel(Event evt)
    {   Checkbox checkbox = (Checkbox)evt.target;   String label = checkbox.getLabel();
        if (label == "Option 1")
            checkbox.setLabel("Changed 1");
    else
        if (label == "Option 2") checkbox.setLabel("Changed 2");
    else
        if (label == "Option 3")   checkbox.setLabel("Changed 3");
        else {
            checkbox1.setLabel("Option 1");
            checkbox2.setLabel("Option 2");
            checkbox3.setLabel("Option 3");
        }
    }
}

```

3- Dado el siguiente código Java para un applet, construya el modelo de diseño dinámico a través de un Diagrama de Comunicación correspondiente. Tenga en cuenta que se ejecutarán automáticamente los métodos “init” y “paint” en ese orden, enviándosele los mensajes correspondientes a la clase “Applet20”.

```

import java.awt.*;
import java.applet.*;
import MySubClass;

public class Applet20 extends Applet {
    MySubClass mySubObject;
    TextField textField1;

    public void init() {
        mySubObject = new MySubClass(1);
        textField1 = new TextField(10);
        add(textField1);
        textField1.setText("1");
    }

    public void paint(Graphics g) {
        String s = textField1.getText();
        int value = Integer.parseInt(s); //método de clase que transforma string en integer
        mySubObject.SetField(value);
        value = mySubObject.GetField();
        s = String.valueOf(value);
    }
}

```

```

        g.drawString("The myField data field", 30, 80);
        g.drawString(s, 90, 125);
    }
}

```

4.- Para el ejercicio 1 del práctico, construir el modelo de diseño dinámico a través de un Diagrama de Secuencia a partir de la invocación del mensaje “valueForPathChanged”.

Considerar en el modelo los siguientes elementos según corresponda:

- a. considere los retornos y los valores de retornos de los mensajes.
- b. Condiciones e iteraciones.

5- a) Aplicando ingeniería inversa, construir el Modelo de Diseño Dinámico a través de un Diagrama de Secuencia partiendo de la ejecución del método “main”.

b) Aplicando ingeniería inversa, construir el Modelo de Diseño Dinámico a través de un Diagrama de Comunicación partiendo de la ejecución del método “main”.

```

public class AdapterWrapperPattern {

    public static void main(String args[]){
        Guitar eGuitar = new ElectricGuitar();
        eGuitar.onGuitar();
        eGuitar.offGuitar();
        Guitar eAGuitar = new ElectricAcousticGuitar();
        eAGuitar.onGuitar();
        eAGuitar.offGuitar();
    }

    public abstract class Guitar{
        abstract public void onGuitar();
        abstract public void offGuitar();
    }

    public class ElectricGuitar extends Guitar{

        public void onGuitar() {
            System.out.println("Playing Guitar");
        }
        public void offGuitar() {
            System.out.println("I'm tired to play the guitar");
        }
    }

    public class AcousticGuitar{

        public void play(){
            System.out.println("Playing Guitar");
        }
        public void leaveGuitar(){
            System.out.println("I'm tired to play the guitar");
        }
    }
}

```

```

        }
    }

    public class ElectricAcousticGuitar extends Guitar{
        AcousticGuitar acoustic = new AcousticGuitar();

        public void onGuitar() {
            acoustic.play();
        }

        public void offGuitar() {
            acoustic.leaveGuitar();
        }
    }
}

```

Considerar en el modelo dinámico los siguientes elementos según corresponda:

- a. considere los retornos y los valores de retornos de los mensajes.
- b. Condiciones e iteraciones.

6- A partir del siguiente código Java:

```

abstract class EmployeeType {
    abstract int payAmount(Employee emp);
}
class Salesman extends EmployeeType {
    int payAmount(Employee emp) { return emp.getMonthlySalary() +
emp.getCommission(); }
}
class Manager extends EmployeeType {
    int payAmount(Employee emp) { return emp.getMonthlySalary() + emp.getBonus();
}
}
class Engineer extends EmployeeType {
    int payAmount(Employee emp) { return emp.getMonthlySalary(); }
}
class Employee {
    private Set _employeeTypes = new HashSet();

    public static Set getEmployeeTypes() { return _employeeTypes; }
    public void addEmployeeTypes (EmployeeType c) { __employeeTypes.add(c); }
    public void removeEmployeeTypes (EmployeeType c) { _employeeTypes.remove(c);
}
    public int allPayToEmployeeTypes () {
        Set s = Employee.getEmployeeTypes ();
        Iterator iter = s.iterator();
        int total = 0;
        while (iter.hasNext()) {
            EmployeeType each = (EmployeeType) iter.next();

```

```

        total += each.payAmount();
    }
    return total;
}

```

Construir el modelo de diseño dinámico a través de un Diagrama de Secuencia a partir de la invocación del mensaje “allPayToEmployeeTypes”.

Considerar en el modelo los siguientes elementos según corresponda:

- a.- considere los retornos y los valores de retornos de los mensajes.
- b.- Condiciones e iteraciones.

7. Analizar el siguiente código Java.

```

class Motor {
    public void start() { }
    public void rev() { }
    public void stop() { }
    public void service() {
        System.out.println("Motor service");
    }
}

class Rueda {
    public void inflate(int psi) { }
}

class Window {
    public void rollup() { }
    public void rolldown() { }
}

class Puerta {
    public Window window = new Window();

    public void open() { }
    public void close() { }
}

public class Auto {
    public Motor motor = new Motor();
    public Vector ruedas = new Vector();
    public Puerta izq = new Puerta(), der = new Puerta();

    public Auto() {
        for (int i = 0; i < 4; i++)
            ruedas.add(new Rueda());
        motor.service();
        for (int i = 0; i < 4; i++)
            ruedas.elementAt(i).inflate(32);
    }
}

```

```

        public static void main(String[] args) {
            Auto auto = new Auto();
        }
    }

```

- a) Aplicando ingeniería inversa construir el Modelo de Diseño Dinámico a través de un Diagrama de Comunicación a partir de la invocación del método “main()”.
- b) Aplicando ingeniería inversa construir el Modelo de Diseño Dinámico a través de un Diagrama de Secuencia a partir de la invocación del método “main()”.

Considerar en el modelo dinámico los siguientes elementos según corresponda:

- a. considere los retornos y los valores de retornos de los mensajes.
- b. Condiciones e iteraciones.

8. Analizar el siguiente código Java.

```

class Main{
    public static void main(String[] args){
        MedioDePago pagoConTarjeta = new Tarjeta();
        pagoConTarjeta.generarPago(44d,3,1);
    }
}
abstract class MedioDePago{
    protected List pagos = new ArrayList();
    public abstract Boolean generarPago(Double monto, Integer cuotas, Integer
usuario);
}
class Pago{
    private Double _monto;
    private Integer _usuario;
    private Double _interes;
    private Double _descuento;
    private Integer _cuotas;

    public void setMonto(Double monto){ _monto = monto; }
    public void setUsuario(Integer usuario){ _usuario = usuario; }
    public void setInteres(Double interes){ _interes = interes; }
    public void setDescuento(Double descuento){ _descuento = descuento; }
}
    public void setCuotas(Integer cuotas){ _cuotas = cuotas; }
    public Double getMonto() { return _monto; }
    public Double getUsuario(){ return _usuario; }
    public Double getInteres(){ return _interes; }
    public Double getDescuento(){ return _descuento; }
    public Integer getCuotas(){ return _cuotas; }
}
class Tarjeta extends MedioDePago{
    private static Double TASA_INTERES_10 = 0.1d; //10% de interes

```

```

private static Double TASA_INTERES_5 = 0.05d; //5% de interes

public Boolean generarPago(Double monto, Integer cuotas, Integer usuario){
    Pago p = new Pago();
    if(cuotas <= 3){
        p.setMonto(monto);
        p.setUsuario(usuario);
        p.setCuotas(cuotas);
        p.setInteres(monto * TASA_INTERES_5);
    }
    else {
        p.setMonto(monto);
        p.setUsuario(usuario);
        p.setCuotas(cuotas);
        p.setInteres(monto * TASA_INTERES_10);
    }
    pagos.add(p);
}
}

```

Aplicando ingeniería inversa, construir el Modelo de Diseño Dinámico a través de un Diagrama de Secuencia a partir de la invocación del mensaje “generarPago()” de la clase “Tarjeta”.